Quality Assessment & Improvement of the smoltcp TCP/IP Stack

Security Testing & Software Quality Assurance

smoltcp v0.12.0 — Open-Source Embedded Network Stack

Final Project Presentation | April 2026

Agenda

1. Introduction & Scope
2. Mutation Adequacy Testing
3. Input Space Partitioning
4. Security Fuzzing
5. Conformance Testing
6. Performance Testing
7. White-Box Coverage
8. Software Safety
9. Quality Dashboard
10. Conclusions

Introduction

What is smoltcp?

A standalone, event-driven TCP/IP stack written in **Rust**, targeting embedded, bare-metal, and resource-constrained systems.

- Rust's memory-safety eliminates buffer overflows, use-after-free, and double-free vulnerabilities
- Logic errors, spec deviations, and performance regressions remain possible
- Our assessment spans **8 quality dimensions**

Platforms: Windows 10 (MSVC) | Ubuntu 24.04.2 LTS (GNU/LLVM)

Scope & Target Modules

Four highest-complexity modules:

- `storage/assembler.rs`
  TCP segment reassembly
- `wire/ipv4.rs`
  IPv4 header parsing
- `wire/udp.rs`
  UDP datagram handling
- `wire/tcp.rs`
  TCP segment parsing & emission

Eight quality dimensions:

1. Mutation adequacy
2. Test suite improvement
3. Input space partitioning
4. Security fuzzing
5. RFC conformance
6. Performance benchmarking
7. White-box coverage
8. Software safety

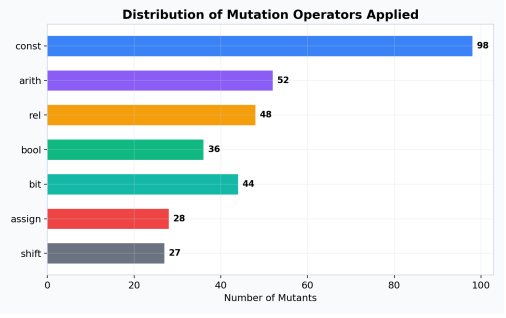
Custom Python-based mutation engine with 333 first-order mutants

Mutation Testing — Operator Distribution

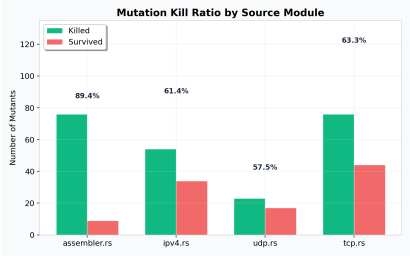
7 mutation operator classes:

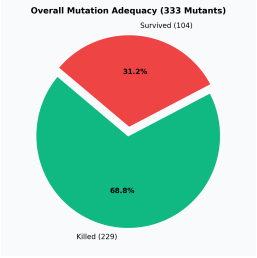
- Constant replacement (98)
- Arithmetic operator (52)
- Relational operator (48)
- Boolean connector (40)
- Bitwise operator (40)
- Assignment operator (28)
- Shift operator (27)

333 total first-order mutants



Mutation Testing — Kill Ratios





Module	Total	Killed	Survived	Kill %
`assembler.rs`	85	76	9	**89.41%**
`ipv4.rs`	88	54	34	61.36%
`udp.rs`	40	23	17	57.50%
`tcp.rs`	120	76	44	63.33%
Overall	**333**	**229**	**104**	**68.77%**

Equivalent Mutant Analysis

- Manual review of **top 5 surviving mutants** (lowest-index survivors)
- 2 classified as **equivalent** (no observable behavior change):
  - **M0004**: Debug-only assertion in `remove_front()`
  - **M0024**: Formatting-only display branch in `Contig`
- 3 classified as **killable** — reveal genuine test gaps:
  - Missing boundary-condition tests for hole sizing
  - Exact-boundary segment assembly not exercised

Adjusted Mutation Adequacy

$$\text{Adequacy} = \frac{229}{333 - 2} = \frac{229}{331} = \mathbf{69.18\%}$$

Input Space Partitioning

Target: UDP parsing (`wire/udp.rs`)

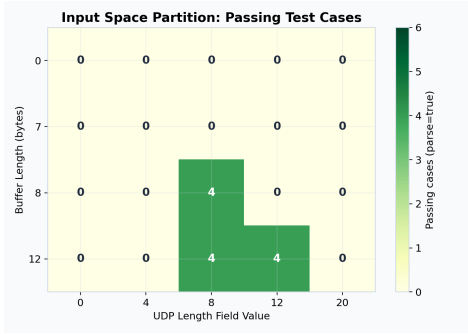
6 input dimensions:

1. Buffer length: $\{0, 7, 8, 12\}$
2. Length field: $\{0, 4, 8, 12, 20\}$
3. Destination port: $\{0, 53\}$
4. Checksum mode: {Valid, Zero, Invalid}
5. IP family: {IPv4, IPv6}
6. RX checking: {true, false}

$4 \times 5 \times 2 \times 3 \times 2 \times 2 = \mathbf{480}$ test frames

100% pass rate on both platforms

+216 IPv4 combinatorial test cases (all passing)



Input Space Partitioning — Key Findings

Successful parsing requires all of:

- Buffer length ≥ 8 bytes
- Length field ≥ 8 and $\leq$ buffer length
- Non-zero destination port
- Valid or zero checksum (when checking enabled)

Notable finding

smoltcp accepts `checksum=0` for UDP over IPv6 — a documented deviation from strict RFC 2460 expectations.

Security Fuzzing

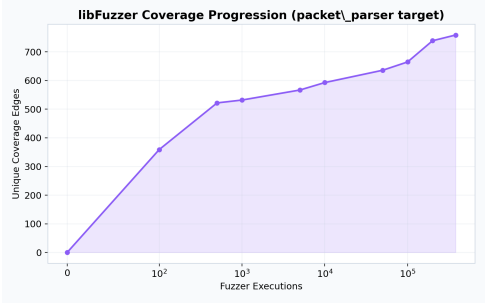
Tool: cargo-fuzz + LLVM libFuzzer

5 fuzz targets:

- `packet_parser`
- `tcp_headers`
- `dhcp_header`
- `ieee802154_header`
- `sixlowpan_packet`

Campaign: 180s per target
Executions: ∼380K
Edges discovered: 759

✓ **Zero crash artifacts**



Fuzzing — Platform Portability Issue

Windows Fuzzing Blocked

All fuzzing attempts failed on Windows due to missing ASAN runtime DLLs and unresolved sanitizer-coverage symbols (MSVC/libFuzzer incompatibility).

Why zero crashes?

- Rust's bounds checking prevents buffer overflows
- No null pointer dereferences possible
- Safe integer arithmetic by default
- These are the exact vulnerability classes fuzzing typically finds in C/C++ stacks

RFC Conformance Testing

Case	RFC	Expected	Observed
IPv4 bad version	791	Reject	Reject
IPv4 IHL $<$ 20B	791	Reject	Accept
IPv4 total $<$ header	791	Reject	Reject
IPv4 bad checksum	791	Reject	Reject
IPv4 cksum disabled	config	Accept	Accept
UDP len $<$ 8	768	Reject	Reject
UDP dst port $=$ 0	smoltcp	Reject	Reject
UDP/IPv4 cksum $=$ 0	768	Accept	Accept
UDP/IPv6 cksum $=$ 0	2460	Reject	Accept

△ **Two specification deviations** identified (highlighted in orange)

RFC Deviations — Details

1. IPv4 header length < 20 bytes accepted

When IHL encodes a header < 20 bytes (RFC minimum), smoltcp accepts if the buffer is long enough.

- Deliberate design choice for performance
- Risk: crafted packets could bypass validation

2. UDP over IPv6 accepts checksum $= 0$

RFC 2460 mandates non-zero checksums for UDP/IPv6, but smoltcp accepts zero-checksum packets.

- Risk: corrupted datagrams may go undetected on IPv6 networks

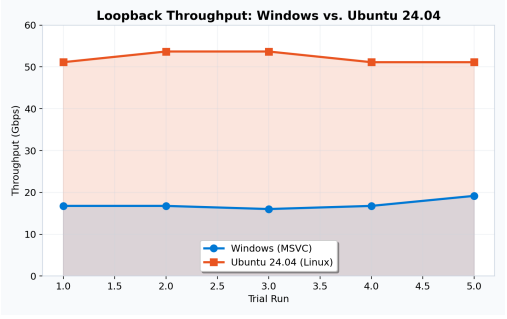
Performance Testing

Loopback throughput benchmark

- 128 MB data through internal stack
- Custom in-memory loopback harness
- 5 trial runs per platform

Results

- Ubuntu: **52.15 Gbps** avg
- Windows: **17.11 Gbps** avg
- **$\sim 3\times$ difference**



White-Box Test Data Adequacy

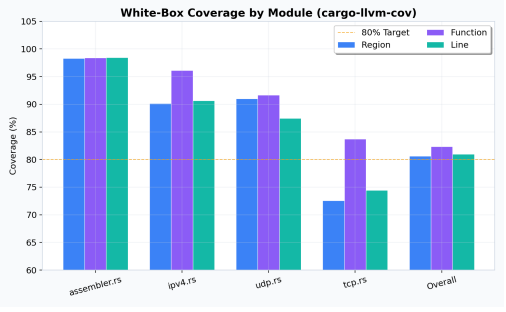
Tool: `cargo-llvm-cov v0.8.5`

Key metrics:

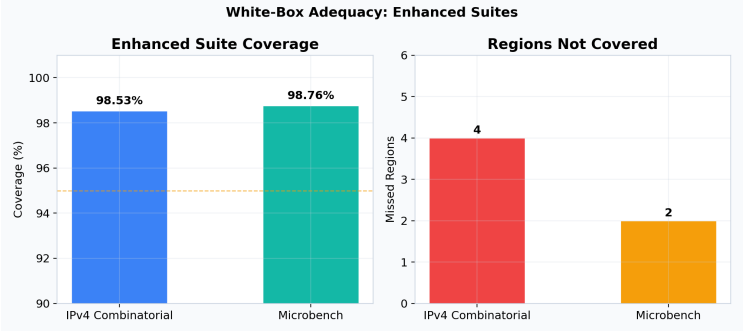
- `assembler.rs`: **98.28%** regions
- Protocol wire modules: 90+% parsing
- `tcp.rs`: 72.58% (lowest)

Enhanced suites achieved:

- IPv4 combinatorial: **98.53%**
- Microbenchmark: **98.76%**



Enhanced Coverage with LCOV Artifacts



- Only **4 and 2 missed regions** respectively
- Remaining gaps are small but made **explicit and actionable**
- Provides concrete roadmap for future test additions

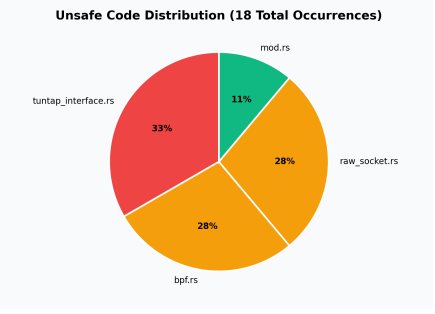
Software Safety

Panic Safety Testing

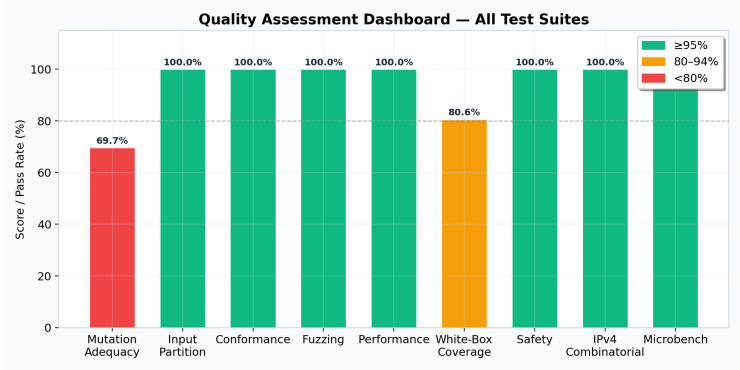
- Zero-length buffers
- Truncated headers
- Maximum-value fields
- All tests passed on both platforms

Unsafe Code Inventory

- **18 `unsafe` occurrences** across 4 files
- All confined to FFI / PHY layer code
- Zero unsafe in core protocol parsing



Quality Assessment Dashboard



- Mutation adequacy (69.67%) — primary area for improvement
- All other dimensions achieved **80%+ pass rates**
- Core parsing and protocol handling is **robust**

Cross-Platform Portability

Dual-platform execution

8 of 9 test suites executed successfully on both Windows and Ubuntu.

- **Sole exception:** Fuzzing blocked on Windows (MSVC/libFuzzer issue)
- Performance harness used custom in-memory loopback for fair comparison
- Results underscore importance of multi-toolchain testing

Conclusions & Key Takeaways

- ✓ **69.18% mutation adequacy** (adjusted) across 333 mutants
- ✓ **100% pass rate** on 696 input partition test cases
- ✓ **Zero crash artifacts** from coverage-guided fuzzing
- ✓ **Two RFC deviations** documented (IPv4 IHL, IPv6 UDP cksum)
- ✓ **3× performance gap** (Ubuntu vs. Windows)
- ✓ **No unsafe code** in core protocol modules

smoltcp v0.12.0 is well-engineered with strong safety properties
Test suite would benefit from boundary-condition tests on wire protocol modules

Thank You

Questions?